Project Executable Files

Date	5 July 2026
Team ID	XXX
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Dataset and trained model files	Yes
5	Environment configuration file (.env.example or similar)	N/A
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	N/A
8	Dockerfile / Containerization config (if applicable)	N/A
9	Test files and test results	Yes
10	Demo video or walkthrough	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
OptiCrop/
├── app/
│   ├── app.py
│   ├── static/
│   │   ├── script.js
│   │   ├── style.css
│   │   └── images/
│   └── templates/
│       └── index.html
├── dataset/
│   └── Crop_recommendation.csv
├── model/
│   └── crop_model.pkl
├── notebook/
│   └── OptiCrop.ipynb
├── playwright-report/
├── Project Documentation/
│   ├── Brainstorming & Ideation
│   ├── Project Demonstration
│   ├── Project Design Phase
│   ├── Project Development Phase
│   ├── Project Documentation
│   ├── Project Planning Phase
│   ├── Project Testing
│   └── Requirement Analysis
├── test-results/
├── tests/
│   └── opticrop.spec.js
├── .gitignore
├── package-lock.json
├── package.json
├── playwright.config.js
├── Procfile
├── README.md
└── requirements.txt
```

```
│
├── static/
│   ├── css/
│   ├── js/
│   └── images/
│
├── templates/
│   └── index.html
│
├── main.py
├── requirements.txt
├── README.md
├── .env
├── .gitignore
│
└── assets/
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://opticrop-h1pt.onrender.com/
Login Credentials (Demo)	No Login Required
Platform / Hosting Provider	Render
Repository Link	https://github.com/sivaeeduri01/OptiCrop-Smart-Agricultural-Production-Optimization-Engine
Demo Video Link	https://drive.google.com/file/d/1j21qATbvw5oR1h6nIJ24FInhlu0EGnVB/view?usp=sharing

https://drive.google.com/file/
1j21qATbvw5oR1h6nIJ24FI

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites
Python 3.x
pip package manager
A modern web browser
Git

Step 1
Clone the OptiCrop repository from GitHub.

Step 2
Install dependencies:
pip install -r requirements.txt

Step 3
No API key or .env configuration is required.

Step 4
Run the Flask application:
python app/app.py

Step 5
Open the local URL shown in the terminal, normally http://127.0.0.1:5000/.

Step 6
Enter N, P, K, temperature, humidity, pH, and rainfall values, then click Analyze & Recommend.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	The trained model is limited to crop classes represented in the dataset.	Retrain the model when expanding crop coverage.
2	Prediction confidence depends on the supplied input values and trained model.	Use the result as decision support and combine it with local agronomic advice.
3	The free Render service may take time to wake after inactivity.	Wait for the service to start; later requests are usually faster while active.